

Q1. What are the most common reasons for missing data in ETL pipelines?

Data usually goes missing in ETL pipelines because of these common reasons:

- **Source data wasn't complete**
 - The original system didn't have the data.
 - An API didn't return all records.
 - Some fields were optional and left blank.
- **Extraction failed**
 - The job stopped midway.
 - Network or connection issues.
 - Wrong date filter (so some records weren't picked up).
- **Transformation logic removed it**
 - A filter accidentally excluded valid records.
 - A wrong join (like INNER JOIN) dropped rows.
 - Validation rules rejected some data.
- **Loading errors**
 - Database rejected rows (constraint errors).
 - Table was overwritten accidentally.
 - The job crashed before finishing.
- **Schema changes**
 - A column name or data type changed and the pipeline wasn't updated.
- **Human mistakes**
 - Misconfigured jobs.
 - Incorrect logic or deployment errors.

Q2. Why is blindly deleting rows with missing values considered a bad practice in ETL?

Blindly deleting rows with missing values is a bad practice because:

- Not all missing values mean "bad" data — sometimes they're optional.
- You could remove a large portion of your dataset.

- It can bias your analysis (results become misleading).
- You lose information that might be valuable later.
- It hides underlying data quality problems instead of fixing them.

Q3. Explain the difference between: Listwise deletion Column deletion

Listwise Deletion

You delete the entire row if even one value is missing.

- If a person's record has one empty field, the whole record is removed.
- You lose complete entries of data.

Example:

If a row has 5 columns and 1 is missing → delete the whole row.

Column Deletion

You delete the entire column if it has too many missing values.

- The full variable/field is removed from the dataset.
- All rows lose that specific column.

Example:

If the "Middle Name" column is mostly empty → remove the whole column.

Example scenario:

A "Secondary Phone Number" column is empty for most customers and isn't needed for reporting, so it can be removed.

Listwise deletion → good when few rows are incomplete

Column deletion → good when a column is mostly empty and not important

Q4. Why is median imputation preferred over mean imputation for skewed data such as income?

Median is better for skewed data because it is not affected by extreme values.

For income data, a few people earn very high salaries. These high values pull the mean (average) up.

- Mean gets affected by outliers.
- Median (middle value) does not.

So when filling missing income values, the median gives a more realistic typical value, while the mean may give a value that is too high.

Q5. What is forward fill and in what type of dataset is it most useful?

Forward fill means filling a missing value with the last available (previous) value.

If data is missing, you just copy the value from the row above it.

Example :-

Date	Sales
Jan 1	100
Jan 2	(missing)
Jan 3	120

After forward fill:

Date	Sales
Jan 1	100
Jan 2	100
Jan 3	120

Most useful for Time-series data (data collected over time) . Such as: Stock prices , Sensor readings , Daily sales , Temperature data

Q6. Why should flagging missing values be done before imputation in an ETL workflow?

Flagging missing values before imputation is important because once you fill them, you can't tell which values were originally missing. By marking them first, you preserve information about data quality, allow analysts or models to know which values were estimated, and maintain transparency in your ETL workflow.

Q7. Consider a scenario where income is missing for many customers. How can this missingness itself provide business insight

If many customers don't provide their income, that missing info can still tell the business something.

- Those who skip income might behave differently, like buying cheaper products or being more privacy-conscious.
- By marking "income missing," you can use it in models for marketing, recommendations, or credit decisions.

So even missing data can give useful business insights.

Q8. Listwise Deletion Remove all rows where Region is missing.

1. Tasks: Identify affected rows

(105,'Amit Verma','Pune',18000,58000,'NaN'),

2. Show the dataset after deletion

Customer_	Name	City	Monthly_S	Income	Region
101	Rahul Meh	Mumbai	12000	65000	West
102	Anjali Rao	Bengaluru	NaN	Nan	South
103	Suresh Lye	Chennai	15000	72000	South
104	Neha Singh	Delhi	NaN	NaN	North
106	Karan Shah	Ahmedaba	NaN	61000	West
107	Pooja Das	Kolkata	14000	NaN	West
108	Riya Kapoor	Jaipur	16000	69000	North

3. Mention how many records were lost

One

Q9. Imputation Handle missing values in Monthly_Sales using:

Forward Fill

1. Tasks: Apply forward fill

#update NaN TO NULL

UPDATE customer SET Monthly_Sales = NULL WHERE Monthly_Sales = 'NaN';

UPDATE customer SET Income = NULL WHERE Income = 'NaN' OR Income = 'Nan';

SET @last_sales = NULL;

SET @last_income = NULL;

Alter table

ALTER TABLE customer

MODIFY Monthly_Sales INT,

MODIFY Income INT;

Apply Forward fill

SELECT Customer_ID, Name, City,

CAST(@sales := IF(Monthly_Sales IS NOT NULL, Monthly_Sales, @sales) AS CHAR) AS

Monthly_Sales_Filled,

CAST(@income := IF(Income IS NOT NULL, Income, @income) AS CHAR) AS Income_Filled,

CAST(@region := IF(Region IS NOT NULL, Region, @region) AS CHAR) AS Region_Filled

FROM customer

CROSS JOIN (SELECT @sales := NULL, @income := NULL, @region := NULL) AS vars

ORDER BY Customer_ID;

2. Show before vs after values

Before

Customer_ID	Name	City	Monthly_Sales	Income	Region
101	Rahul Mehta	Mumbai	12000	65000	West
102	Anjali Rao	Bengaluru	NaN	Nan	South
103	Suresh Lyer	Chennai	15000	72000	South
104	Neha Singh	Delhi	NaN	NaN	North
106	Karan Shah	Ahmedabad	NaN	61000	West
107	Pooja Das	Kolkata	14000	NaN	West
108	Riya Kapoor	Jaipur	16000	69000	North

After

Customer_ID	Name	City	Monthly_Sales	Income_	Region
101	Rahul Mehta	Mumbai	12000	65000	West
102	Anjali Rao	Bengaluru	12000	65000	South
103	Suresh Lyer	Chennai	15000	72000	South
104	Neha Singh	Delhi	15000	72000	North
106	Karan Shah	Ahmedabad	15000	61000	West
107	Pooja Das	Kolkata	14000	61000	West
108	Riya Kapoor	Jaipur	16000	69000	North

3. Explain why forward fill is suitable here

Forward fill is suitable here because your customer table has sequential data where missing values in Monthly_Sales, Income, or Region likely indicate unrecorded information rather than zero or unknown values; by carrying the last known value forward, we preserve continuity, maintain consistency for reporting or analysis, and ensure that calculations like totals or averages aren't disrupted by gaps, making the dataset complete and reliable for further use.

Q10. Flagging Missing Data Create a flag column for missing Income.

1. Tasks: Create Income_Missing_Flag (0 = present, 1 = missing)

```
ALTER TABLE Flag
```

```
ADD COLUMN Income_Missing_Flag TINYINT;
```

```
-- Update the flag to 1 if Income is NULL or 'NaN'
```

```
UPDATE Flag
```

```
SET Income_Missing_Flag = IF(Income IS NULL OR Income = 'NaN' OR Income = 'Nan', 1, 0);
```

```
-- Check the result
```

```
SELECT Customer_ID, Name, Income, Income_Missing_Flag
```

```
FROM Flag;
```

2. Show updated dataset

Customer_ID	Name	Income	Income_Missing_Flag	
101	Rahul Mehta	65000	0	
102	Anjali Rao	Nan	1	
103	Suresh Lyer	72000	0	
104	Neha Singh	NaN	1	
105	Amit Verma	58000	0	
106	Karan Shah	61000	0	
107	Pooja Das	NaN	1	
108	Riya Kapoor	69000	0	

3. Count how many customers have missing income

3 customers have missing income

102	Anjali Rao	Nan	1
104	Neha Singh	NaN	1
107	Pooja Das	NaN	1